Техническое задание FlowSketch Версия 1

Учебные программы и функции C++17 на Windows

Версия документа 1.0 · 12 октября 2026 года · рабочее название FlowSketch

Документ определяет первую версию приложения: вставка кода или открытие .cpp, построение схем выбранных функций, режим оформления, локальные проекты и экспорт. Каждое обязательное требование имеет идентификатор и связано с приемкой. В документе описано 160 обязательных требований и 50 семантических приемочных случаев.

Область гарантии - обычные переходы внутри выбранной функции C++17. Свернутые библиотечные операции и вызовы обозначаются явно; неподдерживаемые конструкции не превращаются в готовую схему. Технические эксперименты и измерения приложения выполняются после визуальной концепции до основной разработки.

Содержание

1 Назначение и границы продукта
2 Термины и контракт корректности
3 Поддержка C++ первой версии
4 Правила преобразования операторов
5 Вставка кода и открытие файлов
6 Построение и состояния результата
7 Типы блоков и представление схемы
8 Автоматическое размещение и исправление оформления
9 Рабочие режимы и пользовательские сценарии
10 Визуальная система и motion
11 Шаблоны и индивидуальное оформление
12 Проекты сохранение и восстановление
13 Экспорт SVG PDF и PNG
14 Архитектура и технические контракты
15 Производительность и надежность
16 Установка и поставка
17 Приемка и критерии завершения
18 Порядок разработки и технические проверки
19 Задание для визуальной концепции GPT Image
20 Коды диагностики
Приложение А Семантические приемочные случаи
Приложение Б Подробные примеры преобразования
Приложение В Форматы данных и прослеживаемость
Источники технических решений

1 Назначение и границы продукта

FlowSketch - рабочее название приложения для построения учебных алгоритмических блок-схем по C++. Первая версия предназначена для студентов, преподавателей и пользователей, которым нужно представить отдельную функцию или небольшую программу, оформить схему и включить ее в отчет. Документ задает требования к версии 1 и условия ее приемки.

Основной результат работы приложения - редактируемый проект с исходным кодом, схемами выбранных функций, привязкой блоков к строкам, стилем и координатами, а также экспорт в SVG, PDF и PNG. Построение выполняется локально. Качество схемы оценивается отдельно по соответствию коду, размещению и экспорту.

SCOPE-01 Первая версия поддерживает Windows 10 22H2 x64 и Windows 11 x64. Установщик проверяется на обеих системах. Windows ARM64, x86, macOS, Linux, мобильные устройства и браузерная облачная версия находятся за пределами версии 1.

SCOPE-02 В приложении два основных режима: «Код → схема» и «Оформление». В первом режиме пользователь выбирает источник «Вставить код» либо «Открыть .cpp». Оба источника используют один анализатор и одинаковые правила корректности.

SCOPE-03 Входом служит одна единица трансляции: небольшой .cpp, отдельная функция либо законченный набор операторов с заданным контекстом. Если в файле несколько функций, пользователь выбирает нужную; проект может хранить отдельные схемы нескольких функций этого файла.

SCOPE-04 Полноценный импорт CMake или Visual Studio проектов, анализ всех конфигураций сборки, многопоточных программ, раскрытие библиотек, исполнение C++ и генерация C++ из вручную нарисованной схемы не входят в версию 1.

SCOPE-05 Учетная запись, подписка, облачное хранение, совместное редактирование и обязательный внешний AI API не требуются. GPT Image используется на этапе проектирования внешнего вида и не является частью движка построения схем.

Все формулировки «должно» и требования с идентификаторами являются обязательными для приемки версии 1. Численные показатели в разделе производительности - проектные целевые значения, которые предстоит измерить на реализованном приложении. Документ не утверждает, что программа или перечисленные проверки уже выполнены.

2 Термины и контракт корректности

Термин	Определение
AST	Дерево структуры C++ после разбора компилятором. Содержит функции, операторы, выражения и сведения о типах.
CFG	Граф переходов внутри функции. Используется для проверки ветвлений, циклов и выходов.
Модель алгоритма	Представление действий, решений, циклов, вызовов и завершений с привязкой к исходнику.
Модель схемы	Семантические блоки и связи, полученные из модели алгоритма. Не содержит выбранных цветов и координат.
Сцена	Измеренный текст, размеры фигур, позиции блоков, маршруты стрелок и подписи для холста и экспорта.
Абстракция	Явное объединение выражения, вызова или библиотечной операции в один блок без раскрытия внутренних действий.
Ревизия исходника	Идентификатор состояния кода и контекста. Результат анализа привязан к конкретной ревизии.

COR-01 Схема сохраняет порядок явных операторов и переходы поддерживаемых конструкций внутри выбранной функции при указанном профиле C++17. Вызовы и выражения могут оставаться атомарными в пределах описанных правил абстракции. Это контракт обычного внутрипроцедурного потока, а не доказательство правильности самой программы.

COR-02 Постоянная область сведений о схеме показывает выбранную функцию, C++17, состояние актуальности и покрытие: «Обычные переходы внутри функции. Внутренние пути вызовов, исключения и скрытые действия объектов не раскрыты». Сведения сохраняются в проекте и доступны в экспорте по правилам раздела 13.

COR-03 Каждый блок действия, решения или вызова имеет диапазон исходных строк и точную исходную запись. Служебные узлы начала, окончания, слияния и развернутых циклов помечаются как синтетические и связываются с породившей их конструкцией. Нельзя оставлять пользовательский оператор без блока или зарегистрированного правила группирования.

COR-04 Неподдерживаемая конструкция в выбранной функции блокирует выпуск актуальной готовой схемы этой функции. Допустим просмотр предыдущей схемы с явной меткой «Устарела», но частично построенный граф нельзя выдавать за готовый результат.

COR-05 Смена стиля, шрифта, размеров, расположения и трассировка стрелок не меняют типы семантических узлов, их исходные выражения, порядок действий и набор смысловых ребер. До и после операции сравнивается нормализованная семантическая модель.

COR-06 Достижимость не определяется по догадке или путем выполнения кода. Синтаксически присутствующие ветви сохраняются. Недостижимый участок можно пометить только на основании компиляторных сведений, сохранив доступ к исходнику; скрытие таких участков по умолчанию выключено.

COR-07 Приложение не обещает определять результат алгоритма, завершение цикла, отсутствие ошибок памяти или все возможные значения переменных. Неопределенное поведение C++ и неизвестная цель динамического вызова не превращаются в придуманную ветвь.

COR-08 Для одинакового исходника, контекста и версии анализатора семантическая модель одинакова после исключения технических меток времени и временных путей. Автоматическая раскладка также должна быть воспроизводимой при одинаковом стиле и закреплениях.

3 Поддержка C++ первой версии

Основной профиль - стандартный C++17 без расширений конкретного компилятора. Поддержка проверяется по AST выбранной функции, разрешенным операциям и правилам абстракции. Успешного синтаксического разбора недостаточно для присвоения статуса готовой схемы. Объявления вне выбранного тела могут использоваться для разрешения типов и вызовов, но не означают поддержку их внутреннего поведения.

Конструкция	Правило версии 1
Скалярные типы и enum	Объявления, инициализация, присваивания, арифметика, сравнения и встроенные операторы поддерживаются.
Встроенные массивы и указатели	Операции показываются как записанные выражения. Анализ значений указателей и корректности доступа не выполняется.
if и else	Включая вложенные условия, else if, пустую ветвь и init statement C++17.
while do while for	Поддерживаются обычные, вложенные и бесконечные циклы, пропущенные части for.
switch	Поддерживаются case, default, fallthrough, несколько меток и отсутствие default.
break continue return	Переходы определяются с учетом вложенности цикла и switch.
Встроенные && и ||	В условиях сохраняется короткое замыкание; подробное представление показывает отдельные проверки.
Условный оператор ?:	В простом выражении показывается точная запись атомарно; отдельная детальная декомпозиция значений не входит в v1.
Функции и рекурсия	Вызов атомарный. Для доступного поддерживаемого тела открывается отдельная схема без рекурсивного разворачивания.
std::cin и std::cout	Распознаются по разрешенным типам и символам стандартных потоков; изображаются атомарным вводом или выводом.
std::string std::vector и стандартные алгоритмы	Разрешены через профиль библиотечных абстракций: операции остаются атомарными; результат имеет явную метку абстракций.
Комментарии и форматирование	Скобки и ключевые слова внутри комментариев и строк не влияют на структуру.

Конструкция	Правило версии 1
Макросы значений	Разрешены после раскрытия, с сохранением места использования и раскрытия. Макросы управления потоком отклоняются.
range for	Для встроенных массивов, vector<int>, vector<double>, vector<char> и string - профилированная итерация с явной абстракцией.
if constexpr	Не входит в матрицу v1; приложение выдает диагностику ограничения.

CPP-01 Для скалярного ядра поддерживаются void, bool, char, целочисленные и вещественные типы, enum, встроенные массивы и обычные указатели или ссылки. unsigned и const сохраняются в исходных выражениях. Типы не подменяются при визуализации.

CPP-02 std::string и разрешенные std::vector допускаются только с поставляемым профилем стандартной библиотеки. Конструкторы, деструкторы, методы, перегруженные операции и стандартные алгоритмы остаются атомарными и перечисляются в сведениях об абстракциях. Такой результат получает статус «Построено с абстракциями».

CPP-03 Пользовательские классы с пользовательскими конструкторами, деструкторами и перегруженными операторами не входят в проверенный профиль v1. Простые данные struct также не считаются поддержанными автоматически: до отдельного тестового профиля операции с ними отклоняются. Их объявления могут присутствовать во внешнем контексте, если выбранная функция ими не пользуется.

CPP-04 Объявленные функции и вызовы через указатель с разрешенными скалярными сигнатурами показываются атомарным блоком с меткой абстракции. Неизвестная динамическая цель указывается явно. Приложение не выбирает одну цель вызова и не разворачивает несуществующее тело. Виртуальные вызовы пользовательских классов не входят в профиль v1 по CPP-03.

CPP-05 try, catch, throw, goto, coroutines, asm, setjmp и longjmp, препроцессорные схемы нескольких конфигураций, пользовательские зависимые шаблоны и распознанные примитивы управления потоками исполнения отклоняются с конкретной строкой и кодом ограничения.

CPP-06 Факт отсутствия try не означает отсутствия возможных исключений. Нормальный возврат атомарного вызова является выбранной абстракцией. Возможные исключения библиотеки, разрушение объектов и поведение внутри вызова всегда относятся к явно указанной границе покрытия.

CPP-07 Неизвестный AST-узел в выбранном теле не пропускается. Анализатор возвращает диагностируемое ограничение, включая название конструкции и диапазон строк. Тесты подтверждают разрешенные узлы по белому списку, а не только запрещенные ключевые слова.

CPP-08 Внешние объявления и заголовки, необходимые для разбора, отделены от выбранного тела. Ошибка единицы трансляции, мешающая надежному разрешению выбранной функции, блокирует готовую схему. Восстановленное после ошибки AST не используется как подтвержденный результат.

CPP-09 Профиль v1 допускает std::string и std::vector<int>, std::vector<double>, std::vector<char> со стандартными allocators: создание по умолчанию, списку скаляров, размеру и начальному значению, копирование и присваивание; size, empty, clear, push_back,

pop_back, at, operator[], begin и end. Для string дополнительно разрешены строковый литерал, +=, + и сравнения с string или литералом. Вложенные контейнеры, пользовательский allocator и vector<bool> отклоняются. Границы доступа, reallocations и исключения внутри операций не раскрываются.

CPP-10 std::sort, std::reverse, std::min, std::max, std::abs, std::sqrt и std::pow с разрешенными скалярными аргументами или итераторами профилированных контейнеров допускаются атомарно. std::sort используется без пользовательского comparator. Расширение списка требует отдельного профиля и тестов. Любой свернутый вызов, включая скалярную функцию, рекурсию и I/O, переводит результат в статус с абстракциями.

CPP-11 new и delete, volatile и thread_local, локальные static объявления и пользовательская динамическая глобальная инициализация не входят в профиль v1. Глобальные скалярные данные с постоянной инициализацией допускаются как видимый контекст; их начальные значения доступны в сведениях, а не изображаются повторным действием каждой функции. Скрытые выделения памяти внутри профилированной STL остаются заявленной абстракцией.

CPP-12 Для v1 макрос значений означает object-like чистое постоянное выражение без побочных эффектов, &&, ||, ?: и управляющих операторов. Function-like и управляющие макросы отклоняются. Препроцессор разбирается в одной видимой конфигурации defines; разные #if ветви не объединяются. Spelling и expansion locations сохраняются раздельно.

4 Правила преобразования операторов

SEM-01 Объявление с инициализацией, присваивание и обычное выражение образуют блок процесса с точной исходной записью. Несколько соседних простых операторов допускается объединить только при включенном компактном представлении; объединенный блок хранит все исходные диапазоны.

SEM-02 Условие if образует ромб и ребра «Да» и «Нет». Отсутствующая else ветвь ведет к следующему оператору. Ветви с return или другим завершением не соединяются искусственно с общей точкой продолжения.

SEM-03 while проверяет условие до первого выполнения тела. do while выполняет тело до первой проверки. Обратное ребро и выход различимы; do while с continue ведет к нижней проверке.

SEM-04 for разворачивается в инициализацию, проверку, тело и изменение. Инициализация выполняется один раз. Изменение ведет к проверке. Отсутствующее условие означает бесконечный цикл без обычной ложной ветви; выходы через break или return сохраняются.

SEM-05 continue ведет к ближайшему содержащему циклу: к изменению в for, к условию в while, к нижнему условию в do while. Наличие switch внутри цикла не меняет цель continue.

SEM-06 break выходит из ближайшего switch или цикла, а не из всех внешних конструкций. Цель определяется по AST-вложенности и проверяется по нормализованным переходам.

SEM-07 switch вычисляет управляющее выражение один раз. Ребра подписаны case значениями и default; если default нет, остается выход для несовпадения. Fallthrough ведет в следующий case. Несколько case к одному действию не дублируют это действие. break завершает только ближайший switch.

SEM-08 return показывает возвращаемое выражение и завершает функцию. Следующего ребра к обычному оператору нет. Падение в конец void функции показывает обычное окончание. Конец main без явного return отображается синтетическим «Возврат 0» с

привязкой к завершению main. Диагностируемое достижение конца другой функции с возвращаемым значением блокирует готовую схему.

SEM-09 Встроенные && и || в управляющих условиях сохраняют условное вычисление операндов. При подробном представлении && выполняет правую проверку только после истинной левой; || - только после ложной. При компактном представлении полное выражение остается в одном ромбе с меткой условного вычисления.

SEM-10 Перегруженные && и || нельзя разворачивать по правилам короткого замыкания встроенных операторов. Для разрешенных библиотечных операций используется атомарное выражение; пользовательские перегрузки отклоняются по CPP-03.

SEM-11 Составное выражение сохраняется атомарно, если его разбиение потребовало бы приписать C++ неустановленный порядок вычисления аргументов или действий. Несколько вызовов внутри такого выражения не рисуются как произвольная последовательность. Подробный режим раскрывает только реализованные и проверенные правила.

SEM-12 Вызов функции хранит ее разрешенную сигнатуру или сведения о неизвестной динамической цели. В стандартном представлении рисуется предопределенный процесс. Переход к собственной схеме функции доступен только для известного поддерживаемого тела и сохраняет возможность вернуться назад.

SEM-13 Цепочка std::cin >> a >> b образует ввод с точной исходной записью; std::cout << ... - вывод. Проверка опирается на типы и символы, а не имя переменной «cin». Пользовательский объект с таким именем не распознается как стандартный ввод.

SEM-14 std::exit и noreturn учитываются только по разрешенному реальному символу или атрибуту. Невозвращающийся вызов ведет в терминальное состояние «Невозврат из вызова» без обычного ребра продолжения; std::exit отдельно обозначает завершение программы. Нельзя назвать произвольный noreturn возвратом функции или гарантированным завершением всей программы.

SEM-15 Названия блоков по умолчанию содержат исходные выражения. Приложение не заменяет произвольный цикл выдуманным описанием «Сортировка» или «Вычисление суммы». Пользователь может добавить отдельный поясняющий заголовок, не скрывая доступ к точному исходнику.

SEM-16 Range for по разрешенному массиву, vector или string изображается как получение диапазона один раз, проверка наличия следующего элемента, получение элемента, тело и переход итерации. continue ведет к переходу итерации, break - к выходу. Скрытые begin, end, сравнение и изменение итератора объединены в подписанные абстракции; исходная форма range for доступна всегда. Пользовательские диапазоны отклоняются.

SEM-17 В свойствах блока доступен отдельный список абстракций. I/O, библиотечные операции, range for и свернутые вызовы имеют одинаковое правило статуса «Построено с абстракциями». Точная запись составного скалярного выражения не называется раскрытием всех промежуточных вычислений.

SEM-18 Модель сохраняет AST-структуру условных выражений даже при компактном визуальном блоке: для &&, || и ?: фиксируется условное вычисление операндов. Для выражений с нефиксированным порядком аргументов хранится признак такого порядка; они остаются атомарными. Визуальное сворачивание не означает вычисления обоих операндов ?: или последовательности всех вызовов.

5 Вставка кода и открытие файлов

IN-01 Вставка поддерживает целую .cpp программу, определения функций и законченный набор операторов. Пользователь явно выбирает «Программа или функции» либо «Фрагмент операторов». Автоматическое предложение режима допустимо, но применение оболочки показано до построения.

IN-02 Фрагмент операторов помещается в видимую функцию оболочки. Контекст содержит выбранный тип результата, параметры, локальные объявления и разрешенные заголовки. Оболочка не появляется на самой учебной схеме, а ее строки корректно преобразуются обратно в строки пользовательского фрагмента.

IN-03 Если неизвестны имя, тип или заголовок, приложение выделяет место и предлагает задать контекст. Оно не вставляет int, include или заглушку функции без явного действия пользователя. break вне цикла и return с неподходящим типом оболочки остаются ошибкой контекста.

IN-04 Открытие .cpp доступно через кнопку, Ctrl+O и перетаскивание файла. Открытый исходник становится снимком проекта. Правки в редакторе не перезаписывают внешний .cpp. Отдельное действие «Сохранить исходник как» сохраняет копию; перезапись существующего файла требует системного подтверждения.

IN-05 UTF-8 с BOM и без BOM, LF и CRLF поддерживаются. Если байты не образуют UTF-8, пользователь выбирает кодировку, включая Windows-1251, и видит предпросмотр. Молчаливая замена поврежденных символов запрещена. Исходная кодировка и окончания строк сохраняются как метаданные.

IN-06 Версия 1 допускает не более 256 KiB исходника и 5000 строк, не более 500 семантических блоков и 1000 связей на функцию, глубину управляющей вложенности до 24. Ограничения проверяются до передачи на холст; превышение объясняется и не обрезает данные. Это лимиты продукта, а не требования самого C++.

IN-07 Поставляемые стандартные заголовки доступны без настройки. Для собственного соседнего заголовка пользователь явно выбирает разрешенную папку контекста. Рекурсивная загрузка проекта, выполнение скриптов сборки и свободные произвольные флаги компилятора не поддерживаются.

IN-08 Список функций показывает имя, сигнатуру, диапазон строк и доступность схемы. Выбор main предлагается только если main существует и поддерживается. Перегрузки различаются сигнатурами. Выбор каждой функции не теряет исходник и стиль остальных схем.

IN-09 После внешнего изменения .cpp приложение показывает предложение перечитать файл при следующем явном открытии или проверке обновления. Перечитывание не уничтожает несохраненные правки без выбора пользователя. Постоянное слежение за файловой системой не является обязательным для v1.

IN-10 Комментарии сохраняются в редакторе и проекте. Добавление комментария к блоку производится явным действием пользователя; автоматическое превращение каждого комментария в фигуру выключено.

IN-11 Собственные заголовки разрешены только внутри явно выбранной папки контекста. Перед анализом захватывается полный используемый набор текстовых .h и .hpp вместе с транзитивными include: максимум 16 файлов, 512 KiB суммарно, глубина включения 16. Снимки и digest содержимого входят в проект; выход за выбранную папку, цикл include без

стандартной защиты или превышение предела дают диагностику. После открытия проекта используются сохраненные снимки, а обновление с диска выполняется отдельным действием с предпросмотром изменений.

6 Построение и состояния результата

Состояние	Поведение
Нет схемы	Показать доступные способы ввода и один пример. Экспорт недоступен.
Анализ или раскладка	Показать этап, индикатор работы и «Отмена». Редактор остается доступным.
Построено	Схема актуальна; указан профиль обычных переходов и границы покрытия.
Построено с абстракциями	Схема актуальна; видны количество абстракций и раскрываемый список вызовов или библиотечных операций.
Нужно исправить код или контекст	Выделить строки и конкретный способ исправления. Готовый новый результат отсутствует.
Конструкция не поддерживается	Показать название, строку и ограничение v1. Предыдущий результат остается устаревшим.
Устарела	Есть схема другой ревизии. Показать «Обновить схему» и запретить обычный экспорт текущего результата.
Не удалось разместить	Семантический результат сохранен, но качественная сцена не готова. Можно снять закрепления и повторить раскладку.
Превышен лимит или сбой анализатора	Показать причину и действия повторить либо уменьшить вход; исходник и предыдущий проект сохранены.

STATE-01 Построение запускается кнопкой или Ctrl+Enter. Автоматическое построение во время каждого ввода символа выключено по умолчанию. Синтаксическая подсветка редактора не является подтверждением успешного семантического разбора.

STATE-02 Каждое задание получает requestId, sourceRevision, digest исходника и контекста, functionId и версии правил. Изменение кода или контекста помечает зависимые схемы устаревшими. Выбор другой функции переключает ее отдельный результат; кэш прежней функции не устаревает только из-за переключения. Ответ старого задания не заменяет текущую выбранную схему даже если завершился позже.

STATE-03 Изменение только оформления не требует повторного разбора C++; выполняется измерение и раскладка новой сцены. Смена подробности требует проверенного преобразования модели и новой раскладки, сохраняя ревизию кода.

STATE-04 Ошибки и ограничения доступны в панели диагностики с кодом, понятным текстом и переходом к строке. Состояние не обозначается только цветом. Несколько ошибок не скрываются за одной общей надписью «Не удалось».

STATE-05 Для ошибок разбора, неподдерживаемых конструкций и неготовой раскладки обычный экспорт недоступен. Устаревшую схему можно экспортировать лишь отдельным

действием «Экспортировать предыдущую версию» с указанием ревизии и предупреждением в самом экспортированном документе.

STATE-06 Успешный результат никогда не называется «Код проверен» или «Программа корректна». Интерфейс сообщает о построении схемы и конкретном покрытии. Если абстракции есть, их нельзя скрыть сменой шаблона.

7 Типы блоков и представление схемы

Элемент	Использование
Терминатор	Начало выбранной функции и обычное завершение. Возвращаемое значение доступно рядом с выходом.
Прямоугольник	Объявление, присваивание и обычное действие.
Ромб	Управляющее условие с подписанными выходами.
Параллелограмм	Распознанный стандартный ввод или вывод.
Предопределенный процесс	Атомарный вызов функции или библиотечный алгоритм.
Соединитель	Синтетическая точка слияния или подписанная связь между страницами.
Комментарий	Пользовательская поясняющая заметка, не участвующая в исполнении.

VIEW-01 По умолчанию схема строится сверху вниз. «Да» ведет по основному направлению или в левую ветвь по единому правилу выбранного профиля, «Нет» - в противоположную. Выбор стороны не меняет смысл ребра; подписи обязательны.

VIEW-02 Схема функции начинается отдельным терминатором с именем и параметрами. main отображается как функция main; другая выбранная функция не выдается за весь алгоритм программы.

VIEW-03 В версии 1 есть «Подробно» и «Компактно». Подробный режим разворачивает управляющие конструкции по разделу 4. Компактный группирует только последовательные простые действия без условий, вызовов завершения и скрытой потери переходов. Разворачивание группы возвращает отдельные блоки.

VIEW-04 Клик по блоку выделяет исходные строки; клик или выбор строки показывает связанные блоки. Связь поддерживает диапазоны, один оператор в нескольких синтетических блоках и несколько операторов в группе.

VIEW-05 Обзор схемы включает панорамирование, масштаб 10-400 процентов, масштаб 100 процентов, «Показать всю схему» и «Показать выделение». Миникарта появляется только по запросу или для больших схем и не закрывает основные действия.

VIEW-06 Профиль фигур версии 1 описывает обычные учебные алгоритмические схемы. Заявление о сертифицированном соответствии ГОСТ или требованиям конкретного преподавателя не допускается без отдельной спецификации и приемки. Формальный профиль стандарта можно добавить после версии 1.

8 Автоматическое размещение и исправление оформления

LAY-01 Размер блока рассчитывается по фактическому шрифту, переносу, внутренним отступам и форме фигуры. Текст ромба располагается внутри вписанной области; текст параллелограмма учитывает скошенные стороны. Алгоритм не использует ширину прямоугольника как доступную ширину любой фигуры.

LAY-02 Размер основного шрифта схемы задается стилем и не уменьшается автоматически ради размещения. По умолчанию 14 логических px, допустимый диапазон 12-24 px. Выражение переносится, блок растет, а длинные схемы распределяются по пространству или страницам.

LAY-03 Обычный блок имеет ориентир минимальной ширины 160 px и высоты 48 px; внутренний отступ не менее 12 px. Между контурами соседних блоков - не менее 32 px, между параллельными ветвями - 48 px, между несвязанной стрелкой и контуром - 8 px. Значения являются параметрами шаблона с допустимыми нижними пределами.

LAY-04 Стрелки входят в назначенные порты фигур и имеют видимый наконечник. Их маршрут не проходит через несвязанные блоки или текст. Подписи ветвей размещаются рядом со своей стрелкой и не перекрывают другие подписи, контуры и текст.

LAY-05 Маршруты по умолчанию ортогональные. Обратное ребро цикла проходит снаружи его тела и отличается направлением наконечника. При неизбежном пересечении стрелок применяется единый визуальный способ показать отсутствие соединения; соединение существует только в явной точке модели.

LAY-06 Команда «Упорядочить» пересчитывает размещение, трассировку, размеры и подписи текущей схемы, сохраняет семантику и является одной отменяемой операцией. Она не исправляет логические ошибки исходного C++.

LAY-07 Пользователь может закрепить позицию блока. Авторазмещение соблюдает закрепления либо сообщает о несовместимых ограничениях. Оно не выдает столкновение за успешную раскладку. Кнопка «Снять закрепления и упорядочить» действует после явного выбора и доступна для отмены.

LAY-08 После ручного перемещения маршруты связанных стрелок обновляются. Пересечение фигур или текста подсвечивается как проблема оформления; пользователь может отменить действие либо упорядочить. Экспорт проблемной сцены требует исправления геометрических нарушений.

LAY-09 Смена шрифта, размера, подробности, подписи или шаблона запускает переизмерение текста и трассировку. До завершения новая сцена не считается готовой к экспорту. Изменение только цвета не запускает анализ C++.

LAY-10 После повторного построения ручные позиции, комментарии и заголовки переносятся только для однозначно сопоставленных блоков по структуре и исходной идентичности. Одних номеров строк недостаточно. Несопоставленные правки сохраняются в списке и предлагаются для повторного применения; молчаливая потеря запрещена.

LAY-11 Перед экспортом геометрическая проверка выявляет наложения фигур, выход текста за фигуру, стрелки через посторонние блоки и перекрытие подписей. У неизбежного пересечения связей не требуется абсолютного отсутствия пересечений, но обязательно однозначное чтение их связности.

LAY-12 Построение и раскладка не выполняются на главном потоке интерфейса. Во время работы можно отменить задание, изменить код и продолжить панорамирование предыдущей схемы с сохранением статуса ее актуальности.

9 Рабочие режимы и пользовательские сценарии

Основной экран - одно рабочее окно. Верхняя строка содержит название проекта, статус сохранения, два режима и действия сохранения или экспорта. Пользователь не должен открывать отдельное маленькое окно, чтобы постоянно редактировать код и видеть схему.

UX-01 В режиме «Код → схема» слева находятся редактор и выбор источника, справа - холст. Основная кнопка «Построить схему» постоянно доступна рядом с выбором функции. Диагностика раскрывается внизу редактора или отдельной компактной панели.

UX-02 Источники «Вставить код» и «Открыть .cpp» не являются третьим и четвертым основными режимами. После открытия файла он отображается в том же редакторе. Название файла, его снимок и наличие правок видны пользователю.

UX-03 В режиме «Оформление» редактор свернут, холст расширен. Правая контекстная панель содержит настройки схемы целиком или выделенных блоков. Переход обратно сохраняет курсор редактора, выделение, масштаб и позицию холста.

UX-04 Постоянная широкая библиотека всех фигур не требуется. В v1 пользователь меняет оформление автоматически построенной схемы и добавляет комментарии. Добавление или удаление семантических блоков и переподключение логических стрелок вручную не входят в v1.

UX-05 Галерея стилей открывается панелью с предпросмотром на текущей схеме. В ней доступны встроенные, пользовательские, недавние и последний использованный стили. Отмена предпросмотра возвращает прежний стиль; применение - одна операция в истории.

UX-06 Изменение кода или контекста сохраняет старую схему видимой с меткой «Устарела». Обновление не меняет выбранный стиль. При конфликте ручных правок показываются конкретные несопоставленные элементы без автоматического сброса всего проекта.

UX-07 Первый запуск показывает простой пример на 8-12 блоков и действия вставить код или открыть файл. Пример открывается отдельным проектом и не заменяет пользовательский ввод. Развернутый обучающий мастер не обязателен.

UX-08 При ширине окна от 1100 логических px применяется раздельный вид. Ниже редактор и холст переключаются внутри режима, а правая панель открывается поверх второстепенной области. Основные действия доступны при 640×480 логических px; рекомендуемая рабочая область - от 1280×720.

UX-09 При Windows DPI 100, 125, 150 и 200 процентов текст, кнопки, системные диалоги и холст остаются читаемыми; окно не выходит за доступную область. Проверяются 1366×768 при 100 процентах и 1920×1080 при 100, 150 и 200 процентах.

UX-10 Ctrl+N создает проект с проверкой несохраненных изменений; Ctrl+O открывает выбор .cpp или файла проекта; Ctrl+S сохраняет проект; Ctrl+Shift+S сохраняет как; Ctrl+Enter строит; Ctrl+Z и Ctrl+Shift+Z относятся к текущей области редактора или оформления. Escape закрывает панель либо отменяет активное задание по видимому контексту.

UX-11 Интерфейс доступен с клавиатуры. Все действия перетаскивания имеют альтернативу через свойства или клавиши. Фокус виден и не скрывается панелями. Иконки имеют понятные названия и подсказки; ошибки и статусы содержат текст.

UX-12 Закрытие с несохраненными изменениями предлагает сохранить, не сохранять или отменить закрытие. Подтверждения не требуются для выбора стиля, построения, перемещения или других отменяемых действий.

10 Визуальная система и motion

Направление дизайна - спокойный минималистичный редактор с графитовыми поверхностями и холодным сине-фиолетовым акцентом. Объем создается тонкими границами и умеренными тенями. Холст и текст схемы имеют приоритет над декоративными эффектами. Финальная визуальная концепция будет создана после ТЗ через GPT Image; приведенные значения служат исходными токенами, а не утвержденным изображением интерфейса.

Токен	Исходное значение
Темный фон интерфейса	#11151F
Поверхность панелей	#1A2030
Основной текст	#F3F5FA
Вторичный текст	#B4BDD0
Основной акцент	#7C83FF
Дополнительный акцент	#73D5E8
Светлый холст	#F7F8FC
Текст схемы	#172033
Ритм отступов	4 8 12 16 24 32 px
Скругления UI	8-12 px; фигуры схемы определяются своим типом

DES-01 Цвета задаются семантическими токенами, отдельно для UI и схемы. Предусмотрены светлая и темная темы приложения; фон экспортируемой схемы задается ее стилем и не меняется автоматически при переключении темы интерфейса.

DES-02 Исходные шрифты: IBM Plex Sans или другой согласованный шрифт с кириллицей для интерфейса; JetBrains Mono для кода. Для схемы - поставляемый читабельный шрифт с кириллицей. Итоговые начертания и лицензии фиксируются до сборки. Основные элементы UI имеют размер текста 14-16 px, вспомогательные - не менее 12 px.

DES-03 Обычный текст имеет контраст не менее 4.5:1; значимые границы, иконки и фокус - не менее 3:1 к соседней поверхности. Каждая тема проверяется отдельно. Палитра в таблице должна пройти измерение контраста в финальном сочетании цветов.

DES-04 Используется одна семья SVG-иконок с одинаковой толщиной линий. Основные кнопки подписаны текстом. Минимальная область обычного клика - 32×32 логических px, для главных действий - 36-40 px по высоте. Доступные области не перекрываются.

MOT-01 Hover и press обратная связь - ориентир 100-160 ms; раскрытие панели - 180-240 ms; переход между размещениями схемы - 200-350 ms. Значения являются токенами и проверяются по отзывчивости. Бесконечное свечение, движущийся фон и декоративный параллакс в рабочей области не используются.

MOT-02 Анимация объясняет действие: выделение связанного блока, появление панели, смена геометрии. Она прерывается новым действием, не блокирует ввод и не определяет момент логической готовности результата. Для больших схем допускается быстрый переход без движения каждого блока.

MOT-03 Учитывается системное reduced motion; в настройках есть режим «Меньше движения». В нем отключены перемещение и масштабирование как декоративная анимация, а изменение состояния показано сразу или коротким затуханием. Отмена задания и ошибки работают одинаково при включенном и отключенном motion.

11 Шаблоны и индивидуальное оформление

STYLE-01 Встроены четыре стиля: «Учебный» с белым фоном и нейтральными фигурами; «Монохромный» для печати; «Холодный акцент» со сдержанными сине-фиолетовыми оттенками; «Контрастный» с повышенной различимостью линий и текста. Каждый проверяется на одном и том же корпусе схем.

STYLE-02 Стиль хранит семейство и размер шрифта, начертание, цвета фона, текста, заливки и обводки по типам фигур, толщину линий 1-4 px, отступы, ориентиры ширины блоков, расстояния между ветвями и параметры стрелок. Изменение типовой формы решения на обычный прямоугольник недоступно.

STYLE-03 Последний примененный стиль используется в новом проекте. Открытие существующего проекта всегда восстанавливает его собственный стиль. История недавних хранит до 8 разных стилей без дублей. Предпросмотр без применения не изменяет последнее использованное значение.

STYLE-04 Можно сохранить текущие настройки как новый пользовательский стиль, переименовать, дублировать и удалить его. Удаление стиля из библиотеки не меняет проекты, в которых сохранена его копия. Файл стиля .flowstyle содержит версию схемы данных и не содержит исходный C++.

STYLE-05 Доступны общие настройки и исключения для выбранных блоков: цвет, начертание, поясняющий заголовок и позиция. При применении стиля пользователь явно выбирает сохранить или сбросить исключения; по умолчанию они сохраняются.

STYLE-06 В версии 1 выбор ограничен поставляемыми шрифтами с проверенными правами и кириллицей. Неизвестный шрифт импортированного стиля заменяется с уведомлением до объявления сцены готовой; требуется переизмерение. Семейство, начертание и font fingerprint сохраняются в сцене. Экспорт начинается только после готовности шрифтов.

STYLE-07 Смена шаблона, сброс исключений и автоупорядочивание являются отменяемыми действиями. В истории оформления хранится не менее 100 действий или последних 20 MiB записей, в зависимости от достигнутого предела. Исходный код имеет собственную историю редактора.

STYLE-08 Примеры алгоритмов хранятся отдельно от стилей: условие, сумма чисел, поиск максимума, цикл с continue и switch. Они открываются как новые проекты. Название «Шаблоны» в интерфейсе не должно смешивать примеры исходника с оформлением.

12 Проекты сохранение и восстановление

FILE-01 Файл проекта .flowproj - JSON UTF-8 с schemaVersion, projectId, названием, временем сохранения, исходным снимком, режимом ввода, контекстом, sourceRevision, выбранной функцией, схемами, стилями, исключениями, комментариями и закреплениями. Кэш сцен может храниться отдельно и восстанавливаться из модели.

FILE-02 Проект содержит analyzerVersion, semanticRulesVersion, layoutVersion, toolchainProfile и хэш контекста. После изменения несовместимой версии движка старая схема показывается как кэш предыдущей версии до повторного анализа.

FILE-03 Сохранение выполняется через временный файл в той же директории, проверку JSON и предыдущую корректную резервную копию, затем замену целевого файла с обработкой ошибок. Ошибка места на диске, доступа или замены не уничтожает предыдущую читаемую версию. Операция проверяется с искусственными сбоями на каждом этапе.

FILE-04 Автовосстановление записывает отдельный снимок не позднее 5 секунд после прекращения ввода или изменения оформления. При сбое приложение на следующем запуске предлагает восстановить снимок с датой и исходным проектом. Нельзя считать автоснимок явным сохранением исходного .cpp.

FILE-05 Пользовательские данные размещаются в каталоге приложения внутри LocalAppData, а документы - в выбранной пользователем папке. Приложение работает при пути с пробелами и кириллицей. Абсолютный путь исходного файла не является единственным способом восстановить код проекта.

FILE-06 Новая неизвестная schemaVersion открывается с понятной ошибкой и не перезаписывается. Поврежденный файл проверяется до загрузки на холст; доступна попытка открыть последнюю резервную копию. Миграция поддерживаемой старой версии сохраняет оригинал.

FILE-07 Расширение .flowproj можно связать с приложением в установщике. Ассоциация .cpp не меняется автоматически. «Недавние проекты» содержит до 10 путей; недоступный путь обозначается и может быть удален из списка.

FILE-08 При получении новой модели блоки имеют устойчивую внутреннюю идентичность, а sourceRange служит привязкой, не единственным ID. Сопоставление не переносит оформление между неоднозначными одинаковыми операторами без подтвержденного соответствия.

FILE-09 Перед заменой проекта сравнивается digest последней прочитанной или сохраненной версии файла. Внешнее изменение, включая другое окно приложения, вызывает конфликт и выбор сохранить копию или перечитать. Статус «Сохранено» появляется после независимого чтения и проверки записанного файла.

FILE-10 Размер .flowproj ограничен 16 MiB, число сохраненных схем - 32. JSON проверяется по schemaVersion, ссылкам ID, типам и размерам коллекций; NaN, Infinity и неизвестные типы узлов не загружаются. Ограничение не служит поводом обрезать исходник или удалить правки. headerSnapshots сохраняют исходные данные и digest собственных заголовков IN-11.

FILE-11 Каждый исторический результат содержит собственные immutable снимки кода и контекста, digests, functionId и версии движков. Устаревший блок открывает строки предыдущего снимка в отдельном просмотре, не подсвечивает ошибочно текущий редактор. Для каждой функции v1 хранит только текущий результат и один последний корректный предыдущий; предел FILE-10 проверяется до сохранения.

13 Экспорт SVG PDF и PNG

EXP-01 Все форматы строятся из одной проверенной геометрической сцены. Экспорт не зависит от текущего масштаба или видимой области холста и включает выбранную функцию целиком. Экспорт нескольких функций выполняется по одной схеме на отдельную группу страниц PDF либо отдельные файлы.

EXP-02 SVG сохраняет векторные контуры, стрелки и текст, не использует foreignObject для основных элементов и не ссылается на внешние картинки или сетевые шрифты. По умолчанию текст преобразуется в контуры для переносимости; доступна текстовая версия с явным предупреждением о зависимости от шрифтов.

EXP-03 PDF содержит векторные фигуры и читаемый текст с допустимым по лицензии встраиванием шрифта либо контурами. Доступны A4 и A3, книжная и альбомная ориентация, «Вписать на страницу» и «Разбить на страницы». Поля по умолчанию 12 mm.

EXP-04 При разбиении PDF на страницы разрез не проходит через фигуру или ее текст. Пересекающая границу связь получает одинаковые подписанные соединители на соседних страницах. Если отдельная фигура не помещается, пользователь выбирает больший формат или корректирует стиль; молчаливое обрезание запрещено.

EXP-05 В режиме вписывания PDF сохраняется согласованный минимум читаемого текста 9 pt. При невозможности вписать схему без уменьшения ниже этого значения приложение предлагает разбиение на страницы. Предпросмотр показывает реальный масштаб печати.

EXP-06 PNG поддерживает множитель разрешения 1× 2× и 3×, непрозрачный либо прозрачный фон и отступ по умолчанию 24 логических px. Проверяются пределы не более 16384 px по стороне и 64 млн пикселей суммарно; при превышении предлагается SVG или PDF без обрезания изображения.

EXP-07 В предэкспортном окне видны функция, ревизия, формат, размеры или страницы, фон и состояние сцены. По умолчанию добавляется подпись «Обычные переходы внутри функции» и число абстракций. Устаревший экспорт всегда содержит видимое предупреждение о предыдущей ревизии. Для актуальной схемы видимую подпись покрытия можно скрыть явной опцией; сведения сохраняются в метаданных SVG и PDF и сопровождающем PNG текстовом поле.

EXP-08 Диалог сохранения проверяет путь и права. Запись экспорта выполняется через временный файл; неполный файл не остается под именем готового. Существующий файл не заменяется без системного подтверждения. После успеха доступно открыть файл или его папку.

EXP-09 Кириллица, знаки сравнения, кавычки, амперсанд, индексы и длинные выражения совпадают с холстом. Размеры блоков, переносы и подписи стрелок сверяются между SVG, PDF и PNG на одном эталонном проекте.

Векторная сцена исключает зависимость качества документа от скриншота DOM. В официальном примере React Flow экспорт изображения выполняется отдельной библиотекой; для проекта принят самостоятельный контракт векторного экспорта [S6].

EXP-10 Экспорт захватывает неизменяемую сцену с sourceRevision, functionId, styleRevision и sceneRevision. Перед публикацией файла проверяется соответствие выбранному результату. Изменение исходника, выбранной функции или оформления во время обычного экспорта отменяет операцию; временный файл не становится готовым. Отдельный экспорт предыдущей версии явно закрепляет ее ревизию.

EXP-11 Экспорт имеет таймаут 15 секунд и бюджет памяти рабочего задания до 1 GiB; пределы пикселей EXP-06 проверяются до растеризации. Сохраненная сцена содержит font fingerprint, использованные файлы шрифта, glyph позиции и переносы. SVG контуры и PDF embedding используют эти же метрики; повторный независимый перенос текста при экспорте запрещен.

14 Архитектура и технические контракты

Слой	Ответственность
Frontend React TypeScript	Редактор, режимы, свойства, история UI, холст, состояния результата. Предлагаемый редактор - Monaco, подлежит проверке в WebView2.
Tauri и Rust	Файлы, проекты, фоновые задания, проверка входа, запуск строго определенного анализатора, отмена и восстановление.
Clang LibTooling analyzer	AST, разрешение типов, источник, CFG и вывод модели алгоритма с диагностикой. Отдельный процесс.
Преобразование модели	Правила учебных блоков, проверка ребер, группирование, привязка к исходнику.
Раскладка ELK и геометрия	Измерение шрифтов, порты, позиции, маршруты, подписи, проверки сцены. ELK работает в worker.
Общий рендерер сцены	Согласованные контуры и метрики для интерактивного холста, SVG, PDF и PNG.

ARCH-01 Базовый кандидат - Tauri 2, React, TypeScript, Vite, Monaco, React Flow и ELK. Выбор подлежит техническим экспериментам до основной реализации. Конкретные версии и лицензии фиксируются lockfiles и manifest зависимостей; необходимость платной зависимости требует отдельного решения владельца проекта.

ARCH-02 AST и CFG используются совместно: структура исходника определяет учебное представление, а контроль переходов сверяется с нормализованным CFG. Опции CFG, включая исключения и скрытые действия, задаются явно и версионируются. Результат не зависит от неизвестных настроек компилятора по умолчанию.

ARCH-03 Семантическая модель содержит nodes с id, kind, sourceRange, originalText и abstraction; edges с source, target, role и label; diagnostics с code, severity, sourceRange и message; coverage со списком раскрытых и нераскрытых аспектов. Геометрия и стиль хранятся отдельно.

ARCH-04 Протокол - ограниченный JSON UTF-8 по stdin/stdout или эквивалентному локальному каналу. Запрос содержит protocolVersion, requestId, sourceRevision, sourceDigest, contextDigest, functionId, source, context и languageProfile. Ответ повторяет ключи, analyzerVersion, status, модель и диагностику. status различает BUILT_NORMAL_FLOW, BUILT_WITH_ABSTRACTION, INPUT_ERROR, CONTEXT_REQUIRED, UNSUPPORTED, CANCELLED и FAILURE; логи отделены от JSON.

ARCH-05 Frontend обращается к узким Rust-командам analyze, cancel, open, save и export. Backend самостоятельно проверяет аргументы, размер входа, версии и допустимые пути. Нельзя считать разрешения WebView достаточной проверкой собственных native-команд.

ARCH-06 Анализатор запускается по известному пути приложения без свободной shell-команды. Пользовательский C++ разбирается без исполнения, линковки и запуска скриптов сборки. Флаги загрузки compiler plugins, произвольные executables и передача неподтвержденного compile command не допускаются.

ARCH-07 Общий deadline построения от команды до готовой сцены - 20 секунд. Внутри него analyzer ограничен 15 секундами и 1 GiB памяти, измерение и layout - 5 секундами, ответ протокола - 16 MiB. Истечение любого предела дает диагностику. Отмена завершает дерево задания не позднее 1 секунды и не затрагивает другие приложения.

ARCH-08 Одновременно публикуется только один результат на ревизию. Новое задание отменяет или делает устаревшим предыдущее. Поврежденный ответ, несовпавший протокол и сбой процесса не загружают граф на холст.

ARCH-09 Точный состав Clang, ресурсных заголовков и стандартной библиотеки фиксируется toolchainProfile. Пакет должен реально разбирать iostream, vector, string, algorithm и cmath без установленного Visual Studio, SDK или Clang. Способ поставки и notices подтверждаются отдельным экспериментом [S2 S7].

ARCH-10 Приложение не отправляет исходный C++, проекты и пути в сеть. Телеметрия и автоматическое внешнее AI-пояснение отсутствуют в v1. Журнал не содержит полный исходник по умолчанию; диагностический пакет с кодом создается только явным экспортом пользователя.

ARCH-11 Дерево analyzer задания управляется Windows Job Object с ограничением памяти и завершением при закрытии владельца. Это управление ресурсами, а не обещание полной изоляции ОС. Layout имеет отдельный таймаут 5 секунд и отмену. Искусственное зависание не оставляет фоновые процессы после отмены [S12].

ARCH-12 Анализатор использует виртуальную файловую систему с bundled заголовками и захваченными снимками IN-11. include за пределами разрешенного набора дает диагностику. Произвольный абсолютный путь, compiler plugin и response file не расширяют доступ. Кэш зависит от toolchainProfile и digest всего контекста.

ARCH-13 Асинхронные результаты раскладки и упорядочивания публикуются только при совпадении modelRevision, styleRevision, sceneRevision задания и текущего назначения. Изменение свойств во время старой раскладки не может вернуть прежний шрифт или позиции. Отмена охватывает analyzer, layout worker и export worker.

15 Производительность и надежность

Целевой стенд: Windows x64, 4 логических CPU, 8 GiB RAM, SSD, экран 1920×1080. Холодный запуск означает старт после завершения процесса приложения; теплое построение - анализ после готовности окна и движка. Время считается от команды пользователя до готовности проверенной сцены. Для каждого показателя публикуются медиана и p95 по 20 запускам эталонного корпуса.

Показатель	Целевое значение для приемки
Холодный запуск рабочего окна	p95 не более 4 s без загрузки большого проекта
Программа до 200 строк и 50 блоков	Теплое построение p95 не более 2 s
Программа до 1000 строк и 200 блоков	Теплое построение p95 не более 5 s

Показатель	Целевое значение для приемки
Раскладка 200 блоков	p95 не более 1.5 s после готовности модели и шрифтов
Отмена задания	Завершение задания p95 не более 1 s
Реакция на кнопку или ввод	Визуальная реакция p95 не более 100 ms
Панорамирование 200 блоков	Не менее 45 fps на целевом стенде при типовой сцене
RAM приложения и анализатора	Пиково не более 1.5 GiB на корпусе до 200 блоков
Экспорт PNG 2× для 200 блоков	p95 не более 5 s в пределах EXP-06
Автовосстановление	Снимок не позднее 5 s после последнего изменения

NFR-01 Показатели являются требованиями, а не текущими измерениями. Если эксперименты показывают недостижимость, предел или стек меняется до основной реализации с записью решения. Нельзя объявить соответствие по одному быстрому примеру.

NFR-02 При 500 блоках приложение остается управляемым: работают отмена, выделение, обзор и диагностика. Время полного построения ограничено ARCH-07; при невозможности завершить дается явный статус без частичной готовой схемы.

NFR-03 Автоупорядочивание, открытие проекта, восстановление и экспорт проверяются при нехватке места, отказе доступа, отмене и сбое процесса. Пользовательский исходник и последняя корректная сохраненная версия не теряются.

NFR-04 Компоненты интерфейса проверяются при двух темах и четырех масштабах DPI. Контраст, клавиатурные действия и reduced motion являются условиями приемки, а не дополнительными улучшениями после релиза.

NFR-05 Версия приложения, движков и профиля toolchain доступна в «О программе». Диагностические коды стабильны внутри версии протокола. Размер установщика и установленного пакета измеряется и публикуется после упаковочного эксперимента без обещания заранее малого размера.

16 Установка и поставка

PKG-01 Основная поставка - офлайн-установщик Windows x64 NSIS с analyzer, native зависимостями, заголовками и шрифтами. Packaging эксперимент выбирает и фиксирует Evergreen Standalone Installer или Fixed Version WebView2, минимальную версию и порядок обслуживания. Online bootstrapper не удовлетворяет обязательной офлайн-установке. Во время установки и работы загрузка зависимости из интернета не требуется [S1].

PKG-02 Чистые Windows 10 и 11 без Visual Studio, SDK, пользовательского Clang, Node.js и Rust проходят установку и строят эталоны с iostream, vector, string, algorithm и cmath при отключенной сети. Отдельно проверяются отсутствие WebView2 и наличие несовместимой старой версии. Наличие dev tools на машине разработчика не доказывает готовность поставки.

PKG-03 Установщик содержит версию, путь, вариант пользовательской установки и понятную диагностику ошибки runtime. Работа под обычной учетной записью не требует постоянного запуска от администратора.

PKG-04 Поставка содержит notices фактически включенных библиотек, шрифтов и toolchain компонентов. Право на распространение заголовков и встраивание шрифтов проверяется до выпуска. Файлы разработчика с неопределенным происхождением в пакет не включаются.

PKG-05 Удаление программы не удаляет сохраненные пользователем .flowproj и экспорты. Удаление локальных настроек и автоснимков выполняется только по явному выбору. Обновление не меняет схему проекта без правил совместимости FILE-02 и FILE-06.

PKG-06 Цифровая подпись установщика желательна для публичного распространения; стоимость и сертификат определяются отдельным решением. Неподписанная внутренняя сборка не называется публичным подписанным релизом. Автообновление не является обязательной функцией v1.

17 Приемка и критерии завершения

Приемка разделяет проверки будущего приложения и проверку этого документа. Наличие ТЗ, схемы архитектуры или макета не означает прохождение приемки программы. Для готовности версии 1 нужны реализованная Windows-сборка и воспроизводимые результаты всех обязательных групп.

Группа	Условие прохождения
A Семантика	Все положительные случаи приложения A дают ожидаемый нормализованный граф; все отрицательные дают нужную диагностику.
B Привязка к исходнику	Для каждого несинтетического блока восстановлены исходные строки; ошибки указывают пользовательские строки даже внутри оболочки.
C Геометрия	На корпусе нет наложений фигур, выхода текста, стрелок через посторонние блоки и неоднозначных подписей; пересечения связей различимы.
D Оформление	Четыре стиля и допустимые шрифты сохраняют семантический граф; работают последнее использование, исключения, отмена и повторная генерация.
E Проект	Успешны сохранение, чтение, backup и восстановление со сбоями; нет потери исходника и ручных правок.
F Экспорт	SVG PDF PNG совпадают со сценой; проверены кириллица, страницы, прозрачность, крупная схема и устаревшая ревизия.
G Windows поставка	Чистые системы и офлайн-проверка PKG-02 пройдены; зависимости перечислены.
H UX и производительность	Выполнены сценарии, DPI, клавиатура, контраст, reduced motion и целевые измерения раздела 15.

ACC-01 Ожидаемые графы корпуса фиксируются вручную по правилам C++, независимо от выходных данных нашего анализатора. Сверка с Clang CFG является дополнительной проверкой, а не единственным эталоном. Snapshot сгенерированного результата без независимого ожидания не доказывает корректность.

ACC-02 Смена пробелов, переносов, комментариев и имен переменных не меняет управляющую структуру схемы. Изменение условного перехода или return обязано изменить граф. Эти преобразования составляют отдельные смысловые регрессионные проверки.

ACC-03 Семантические тесты сравнивают роли и направления ребер после разрешенного объединения узлов, не абсолютные координаты. Геометрические проверки используют численные границы и визуальный просмотр экспортов; их нельзя заменить тестом наличия картинки.

ACC-04 Результаты приемки содержат версию приложения, toolchain, ОС, аппаратный стенд, список проверок и фактические значения. Открытые ошибки, влияющие на неправильные переходы, потерю данных или установку, блокируют выпуск. Известные незначительные визуальные проблемы перечисляются с решением владельца проекта.

ACC-05 Первый полезный вертикальный срез: открыть .cpp со скалярной функцией if и for, получить связанную со строками схему, сменить стиль, сохранить проект и экспортировать SVG. Этот срез проверяется до расширения матрицы на все конструкции; он не заменяет итоговую приемку.

18 Порядок разработки и технические проверки

Этап	Результат и условие перехода
1 ТЗ и визуальная концепция	Зафиксированные требования и концепты GPT Image для двух рабочих режимов и галереи стилей.
2 Технические эксперименты	Проверены packaged C++ headers, модель переходов, раскладка и общий экспорт. Решения зафиксированы до основной реализации.
3 Вертикальный срез	Выполнен ACC-05 в настоящем Tauri окне.
4 Семантическое ядро	Матрица v1 и отрицательные случаи приложения A проходят независимую проверку.
5 Оформление и проекты	Реализованы стили, исключения, повторное построение, сохранение и восстановление.
6 Экспорт и UI	Готовы PDF PNG SVG, адаптация, keyboard и motion с проверкой.
7 Windows сборка и приемка	Чистая офлайн-установка, все группы A-H и отчет по измерениям.

Обязательные ранние эксперименты: самодостаточная поставка анализатора и стандартных заголовков; AST и CFG для continue, fallthrough и сложных условий; сцена с длинными подписями, ромбами и закреплениями; общий SVG PDF PNG с кириллицей; сохранение и отмена со сбоем процесса. Если кандидат не удовлетворяет обязательному контракту, технология меняется до расширения продукта.

Отложенные возможности: импорт многомодульных проектов и compile_commands, пользовательские диапазоны и современные стандарты, формальный профиль ГОСТ, полноценное ручное изменение логики схем, раскрытие пользовательских классов и шаблонов, исключительные пути, совместная работа и опциональные AI-пояснения. Они не являются скрытыми условиями завершения версии 1.

19 Задание для визуальной концепции GPT Image

Концепция должна показать реальный рабочий редактор, естественный для построения алгоритмов. Рабочее название FlowSketch можно заменить отдельно. Начальный референс пользователя используется как ориентир назначения и состава функций, а композиция оптимизируется под два режима и большой холст.

IMG-01 Подготовить три согласованных изображения: основной режим с кодом и схемой; режим оформления с большим холстом и панелью свойств; галерея встроенных и недавних стилей с предпросмотром. Все изображения используют одну типографику, палитру, иконки и геометрию UI.

IMG-02 Основной макет - Windows desktop окно около 1440×900 с русскими подписями. В нем видны проект, «Код → схема», «Оформление», источник .cpp или вставка, выбор функции, построение, статус актуальности, сохранить и экспорт. Внешний вид минималистичный, графитовый с холодным акцентом и светлым рабочим холстом.

IMG-03 В качестве схемы использовать заранее проверенный пример суммирования с if и for из приложения Б. Правильные фигуры, направления и подписи из ТЗ имеют приоритет над декоративной композицией. Итоговая схема внутри сгенерированного изображения проверяется отдельно; визуальный макет не является эталоном логики.

IMG-04 Motion описывается в сопроводительном перечне состояний: раскрытие панели, подсветка кода и блока, переход после упорядочивания. Поведение, responsive, качество текста и доступность затем проверяются в интерактивном прототипе; растровый концепт не заменяет эти проверки.

Дизайн-этап начинается после выпуска этого ТЗ. Основная реализация приложения следует за просмотром визуальной концепции и техническими экспериментами. В данном документе нет утверждения о готовой реализации интерфейса.

20 Коды диагностики

Коды - минимальный контракт протокола. UI переводит их на понятный русский текст и показывает sourceRange там, где он существует. Ошибки ввода, ограничения профиля, отмена и аварии имеют разные состояния.

Код	Причина	Восстановление
PARSE_ERROR	Некорректный C++ или ошибка заголовка	Исправить выделенный участок.
CONTEXT_REQUIRED	Неизвестное имя в режиме фрагмента	Задать параметры, объявления или заголовки.
UNSUPPORTED_CONSTRUCT	Конструкция вне проверенного профиля	Показать название и строку; не выпускать готовую схему.
UNSUPPORTED_TYPE	Класс или контейнер вне профиля	Показать тип и разрешенный список.
MISSING_RETURN	Возможен конец non-void функции без результата	Добавить корректный return либо исправить управляющую структуру.

Код	Причина	Восстановление
SOURCE_LIMIT	Размер, строки или include превышают лимит	Показать фактическое значение и предел.
GRAPH_LIMIT	Граф или глубина больше лимита	Выбрать меньшую функцию или упростить вход.
ANALYSIS_TIMEOUT	Анализ превысил 15 секунд	Сохранить вход; повторить либо уменьшить контекст.
ANALYSIS_MEMORY	Analyzer превысил предел памяти	Прервать задание и сохранить последний корректный проект.
ANALYZER_FAILURE	Сбой процесса или неверный протокол	Показать диагностику и повторить без перезапуска UI.
LAYOUT_FAILED	Не удалось выполнить геометрический контракт	Снять закрепления или изменить стиль и повторить.
STALE_RESULT	Несовпадение ревизий	Обновить схему или явно выбрать предыдущую версию.
FILE_CONFLICT	Файл изменен другим процессом	Сохранить копию либо перечитать.
FILE_INVALID	Неизвестная версия или поврежденный JSON	Не перезаписывать; предложить backup.
EXPORT_LIMIT	PNG или печатная сцена больше предела	Выбрать SVG, PDF или другой формат страницы.
CANCELLED	Задание отменено пользователем	Вернуться к предыдущему состоянию без сообщения об ошибке.

Приложение A Семантические приемочные случаи

Корпус из 50 случаев - минимальная матрица версии 1. Номер T01 соответствует CPP-001 в рабочем описании корпуса. Здесь зафиксированы ожидания; перед реализацией для каждого случая создается независимый полный эталон графа. Дополнительные проверки I/O, профилированных методов и ошибок не ограничены этим числом.

ID	Входной случай	Результат	Обязательная проверка
T01	Линейные действия и массив	Обычные переходы	Все действия сохранены в исходном порядке.
T02	if else	Обычные переходы	Две ветви соединяются перед продолжением.
T03	if без else	Обычные переходы	Ложная ветвь идет сразу к следующему оператору.
T04	else if с return	Обычные переходы	Вторая проверка только после ложной первой; возвраты завершаются.
T05	if с init statement	Обычные переходы	Инициализация один раз перед проверкой.
T06	while	Обычные переходы	Проверка до первого выполнения тела.
T07	do while	Обычные переходы	Тело до первой проверки.
T08	for	Обычные переходы	Инициализация → условие → тело → изменение → условие.
T09	for без условия	Обычные переходы	Нет придуманного ложного выхода.
T10	continue в for	Обычные переходы	Continue идет к изменению счетчика.
T11	continue в while	Обычные переходы	Continue идет к условию.
T12	continue в do while	Обычные переходы	Continue идет к нижнему условию.
T13	break вложенного цикла	Обычные переходы	Завершается только внутренний цикл.
T14	break switch внутри for	Обычные переходы	Продолжается тело for после switch.
T15	continue switch внутри for	Обычные переходы	Переход к increment внешнего for.
T16	switch case default break	Обычные переходы	Правильные метки выбора и выходы.

ID	Входной случай	Результат	Обязательная проверка
T17	switch fallthrough	Обычные переходы	Переход из case в следующее тело сохранен.
T18	Несколько case одного тела	Обычные переходы	Тело не дублируется.
T19	switch без default	Обычные переходы	Есть путь несовпадения к продолжению.
T20	default в середине	Обычные переходы	Fallthrough из default идет в следующий case.
T21	Вложенный switch	Обычные переходы	Break выходит из ближайшего switch.
T22	return и недостижимый хвост	Обычные переходы	После return нет ребра к хвосту; хвост доступен с меткой.
T23	return b ? 1 : 2	Обычные переходы	Точное условное выражение атомарно; затем завершение.
T24	main без return	Обычные переходы	Синтетический возврат 0 в конце.
T25	void без return	Обычные переходы	Обычное завершение в конце тела.
T26	Конец non void	Ошибка	MISSING_RETURN; готовая схема отсутствует.
T27	&& с побочным эффектом	Обычные переходы	Правая проверка только после истинной левой.
T28	|| с побочным эффектом	Обычные переходы	Правая проверка только после ложной левой.
T29	Смешанные && и ||	Обычные переходы	Приоритет и скобки сохраняют нужные условные пути.
T30	Пользовательский operator&&	Ограничение	UNSUPPORTED_TYPE; встроенное короткое замыкание не подставляется.
T31	Тернарное присваивание	Обычные переходы	Полное выражение атомарно; нет выдуманного порядка.
T32	std cin cout	С абстракциями	Ввод и вывод распознаны по типам; граница покрытия видна.
T33	Переменные cin cout	Обычные переходы	Битовые операции остаются процессами.
T34	std endl и цепочка вывода	С абстракциями	Одна цепочка вывода сохраняет точную запись.
T35	string и vector<int>	С абстракциями	Профиль CPP-09; ctor dtor и методы свернуты.

ID	Входной случай	Результат	Обязательная проверка
T36	Пользовательский RAII	Ограничение	UNSUPPORTED_TYPE даже без try.
T37	Вызов с несколькими аргументами	С абстракциями	combine(g(),h()) атомарен; порядок g и h не придуман.
T38	Рекурсия	С абстракциями	Один блок вызова; нет бесконечного раскрытия.
T39	Известный noreturn	С абстракциями	Нет обычного ребра возврата после stop.
T40	Необъявленная функция	Ошибка	PARSE_ERROR в режиме полной функции.
T41	Ошибка скобки	Ошибка	PARSE_ERROR и пользовательская строка.
T42	Фрагмент с контекстом	Обычные переходы	Оболочка скрыта в схеме; исходные строки восстановлены.
T43	Фрагмент без контекста	Контекст	CONTEXT_REQUIRED без выдуманного int.
T44	Макрос константы	Обычные переходы	Место использования и раскрытия сохранены.
T45	Макрос с if	Ограничение	UNSUPPORTED_CONSTRUCT на вызове макроса.
T46	try throw catch	Ограничение	UNSUPPORTED_CONSTRUCT; готовая схема отсутствует.
T47	Шаблон и if constexpr	Ограничение	UNSUPPORTED_CONSTRUCT.
T48	goto	Ограничение	UNSUPPORTED_CONSTRUCT, метка не превращается в обычный процесс.
T49	C++20 coroutine в C++17	Ошибка или ограничение	Диагностика несовместимого профиля до готовой схемы.
T50	Range for по vector<int>	С абстракциями	Проверка итерации, тело и обратный путь по SEM-16.

Геометрическая матрица дополняет корпус: русский заголовок; условие длиной 120 символов; 8 вложенных условий; несколько return; 3 вложенных цикла; switch с 12 case; 50, 200 и 500 блоков; закрепленные конфликтующие блоки; смена каждого из 4 стилей; минимум и максимум размера шрифта. Проверяются отсутствие наложений, правильность портов и неизменность семантики.

Системная матрица: race A → B → ответ B → ответ A; изменение кода во время экспорта; отмена анализатора и layout; сбой каждого шага сохранения; изменение внешнего .cpp или .h; UTF-8 и Windows-1251; путь с кириллицей; поврежденный и будущий .flowproj; DPI и две темы; отключенная сеть и чистая Windows.

Приложение Б Подробные примеры преобразования

Пример Б1 показывает обычный for с continue. В подробном представлении continue должен вести к ++i. Численный результат программы не вычисляется приложением.

```
int sumOdd(int n) {
    int s = 0;
    for (int i = 1; i <= n; ++i) {
        if (i % 2 == 0) continue;
        s += i;
    }
    return s;
}
```

Узел	Следующий переход
Начало	s = 0 → i = 1 → i <= n
i <= n да	i % 2 == 0
i <= n нет	return s → окончание
i % 2 == 0 да	continue → ++i
i % 2 == 0 нет	s += i → ++i
++i	i <= n

Пример Б2 показывает fallthrough. Для x == 1 выполняются оба присваивания. Для x == 2 - только y += 2. При остальных значениях выполняется default.

```
int choose(int x) {
    int y = 0;
    switch (x) {
        case 1: y += 1; [[fallthrough]];
        case 2: y += 2; break;
        default: y = 9;
    }
    return y;
}
```

Обязательные ребра: выбор 1 → y += 1 → y += 2 → return y; выбор 2 → y += 2 → return y; default → y = 9 → return y. Нельзя объединить case 1 и case 2 так, чтобы потерялось различие входа.

Пример Б3 - фрагмент x += 1; if (x > 3) x = 3;. Без объявления x ожидается CONTEXT_REQUIRED. После явного параметра int x и оболочки void snippet(int x) построение разрешено. Клик по первому блоку должен указывать первую строку пользовательского фрагмента, а не строку служебной оболочки.

Пример Б4 - непрозрачный вызов combine(g(), h()). Даже при доступных объявлениях скалярных функций внутренние действия и исключения не раскрыты; результат имеет статус с абстракциями. Приложение сохраняет выражение атомарно и не рисует произвольный порядок g затем h.

Приложение B Форматы данных и прослеживаемость

Ниже приведен минимальный пример контракта проекта. Значения сокращены для пояснения; рабочая JSON Schema описывает все обязательные поля, допустимые типы и лимиты. Пример не является реализованным файлом проекта.

```
{
  "format": "flowsketch-project",
  "schemaVersion": 1,
  "projectId": "...",
  "source": {"text": "...", "revision": "...", "digest": "..."},
  "inputMode": "translation-unit",
  "context": {"language": "c++17", "headerSnapshots": []},
  "selectedFunctionId": "...",
  "engine": {"analyzer": "...", "rules": "...", "layout": "..."},
  "diagrams": [{"functionId": "...", "coverage": "normal-flow",
    "abstractions": [], "nodes": [], "edges": [],
    "style": {}, "overrides": {}, "pins": {}, "notes": []}],
  "viewport": {"zoom": 1, "x": 0, "y": 0}
}
```

Требования	Группа приемки
COR CPP SEM	A Семантика и B Привязка к исходнику
IN STATE	A Входные случаи и H Сценарии UI
VIEW LAY	C Геометрия и H Навигация
UX DES MOT	H Интерфейс и доступность
STYLE	D Оформление
FILE	E Проект и восстановление
EXP	F Экспорт
ARCH PKG	G Поставка и H Надежность
NFR	H Измерения производительности
IMG	Визуальная концепция перед реализацией

Каждое обязательное требование реализуется в конкретном компоненте и связано с проверкой в итоговом протоколе приемки. Если требование меняется, обновляются его текст, связанные случаи, версия документа и запись решения. Удаление проверки без изменения контракта недопустимо.

Источники технических решений

Первичные источники подтверждают возможности используемых инструментов и семантику языка. Продуктовые лимиты, UI и критерии приемки являются требованиями этого проекта, а не цитатами из документации библиотек. Дата проверки источников 12 октября 2026 года.

S1 Tauri Windows Installer. Установщики Windows и варианты поставки WebView2. https://v2.tauri.app/distribute/windows-installer/

S2 Clang LibTooling. Интерфейс отдельного инструмента и анализ кода из памяти. https://clang.llvm.org/docs/LibTooling.html

S3 Introduction to the Clang AST. Структура исходной программы и информация AST. https://clang.llvm.org/docs/IntroductionToTheClangAST.html

S4 Clang CFG. Внутрипроцедурный граф переходов. https://clang.llvm.org/doxygen/classclang_1_1CFG.html

S5 ELK Layered. Слоистая раскладка, порты и маршруты стрелок. https://eclipse.dev/elk/reference/algorithms/org-eclipse-elk-layered.html

S6 React Flow Download Image. Пример внешней библиотеки экспорта изображения. https://reactflow.dev/examples/misc/download-image

S7 Clang Users Manual. Профили, заголовки и Windows toolchain. https://clang.llvm.org/docs/UsersManual.html

S8 WG21 N4659 C++17 draft. Первичный нормативный ориентир для семантики C++17. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf

S9 Tauri Sidecar. Поставка и запуск внешнего анализатора. https://v2.tauri.app/develop/sidecar/

S10 Tauri Architecture. Веб-интерфейс и Rust backend. https://v2.tauri.app/concept/architecture/

S11 LLVM Developer Policy. Состав лицензирования фактически включаемых LLVM компонентов. https://llvm.org/docs/DeveloperPolicy.html#new-llvm-project-license-framework

S12 Microsoft Job Objects. Контроль ресурсов и жизненного цикла дерева задания Windows. https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects